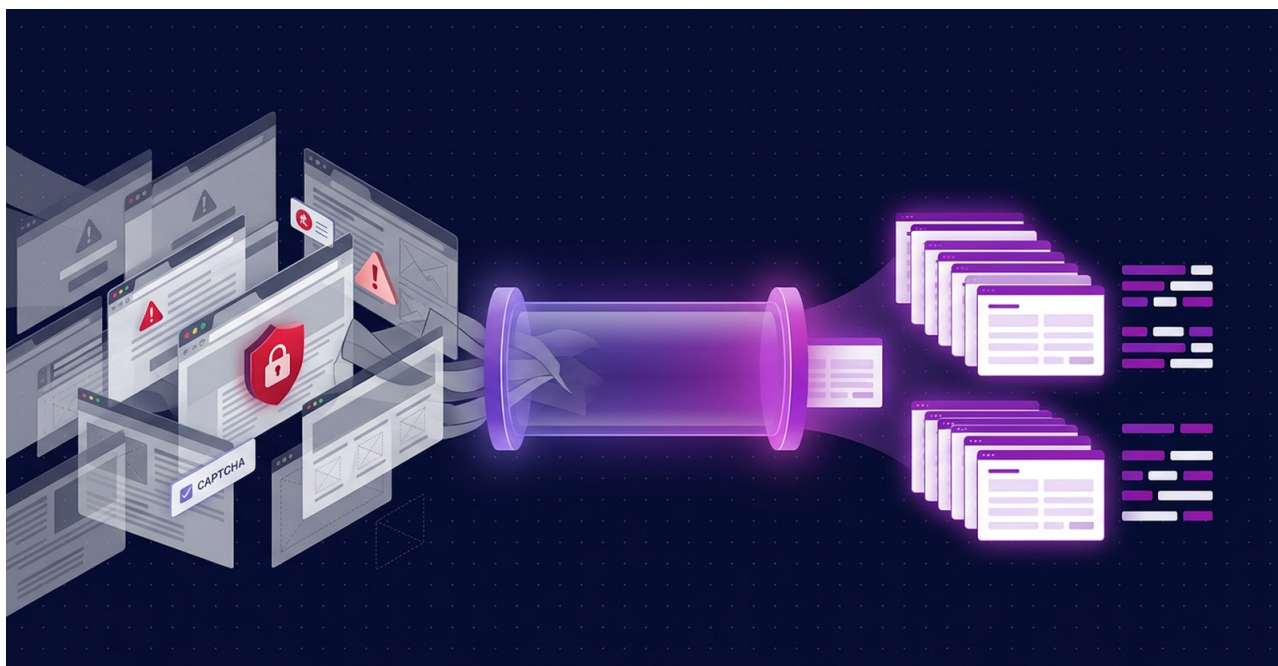


Web Scraping with a Single API: JavaScript Rendering, Unblocking, and Structured Extraction

Web scraping gets a lot easier when it's split into two halves and separate tools are used for each half.



Almost every scraper starts the same way. You open a Python shell, pull down a page with *requests*, parse it with *BeautifulSoup*, pick out the three fields you need, and feel briefly unstoppable. Then you point the same script at a real site and the cracks show up fast. The response comes back as a 403 instead of HTML. The page arrives, but the part you wanted is missing because a JavaScript bundle fills it in after load. Or the scraper runs fine for a week and then returns empty rows one morning because someone on the other side renamed a CSS class. None of these are exotic problems. They are the normal

texture of scraping the modern web, and most of the effort in any real project goes into fighting them rather than into the data you actually care about.

This article is about splitting that work into two clean halves. One half is crawling, which open source tooling handles very well and has for years. The other half is the genuinely hard part: getting past anti-bot defences, running a browser when a page needs one, and turning messy HTML into structured records without hand-writing selectors for every site. That second half is where a single API call earns its place. I will use *Scrapy* for the crawling and the *Zyte API* for the hard

parts, while being honest about where you do not need either.

What makes web scraping hard

Strip away the tooling and scraping comes down to three recurring walls, and the title of this article names all three.

The first is getting blocked. Certain sites can employ a number of techniques to restrict your access to public data. Your script that worked from your laptop starts returning 403 and 429 responses once it runs from a data centre IP, and dodging that reliably means rotating IP addresses, matching browser fingerprints, solving challenges, and respecting per-site quirks. This is a